

Teoria Współbieżności

Laboratorium 1

Wyścig wątków

Kacper Bieniasz

7 października 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Treść zadania

Treść zadania w podpunktach:

1. Napisać program (szkielet), który uruchamia 2 wątki, z których jeden zwiększa wartość zmiennej całkowitej o 1, drugi wątek zmniejsza wartość o 1. Zakładając że na początku wartość zmiennej Counter była 0, chcielibyśmy wiedzieć jaka będzie wartość tej zmiennej po wykonaniu 100000 operacji zwiększania i zmniejszania przez obydwa wątki.
2. Na podstawie 100 wykonań programu z p.1, stworzyć histogram końcowych wartości zmiennej Counter.
3. Spróbować wprowadzić mechanizm do programu z p.1, który zagwarantowałby przewidywalną końcową wartość zmiennej Counter. Nie używać żadnych systemowych mechanizmów, tylko swój autorski.

3 Program w wersji podstawowej

Kod w języku Java (1) przedstawia podstawową wersję rozważanego problemu bez żadnych mechanizmów synchronizacji. Modyfikacja dostarczonego szkieletu programu polegała na uzupełnieniu klasy **IThread** oraz **DThread**, dodanie pętli sterującej tworzeniem i wykonywaniem wątków w klasie **Race**.

```
1 class Counter {
2     private int _val;
3     public Counter(int n) {
4         _val = n;
5     }
6     public void inc() {
7         _val++;
8     }
9     public void dec() {
10        _val--;
11    }
12    public int value() {
```

```

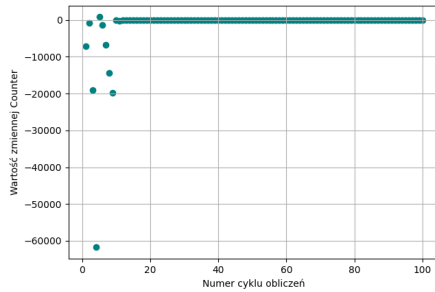
13         return _val;
14     }
15 }
16 // Wątek, który inkrementuje licznik 100.000 razy
17 class IThread extends Thread {
18     private Counter _counter;
19     public IThread(Counter counter) {
20         _counter = counter;
21     }
22     @Override
23     public void run() {
24         for (int i = 0; i < 100_000; i++) {
25             _counter.inc();
26         }
27     }
28 }
29 // Wątek, który dekrementuje licznik 100.000 razy
30 class DThread extends Thread {
31     private Counter _counter;
32     public DThread(Counter counter) {
33         _counter = counter;
34     }
35     @Override
36     public void run() {
37         for (int i = 0; i < 100_000; i++) {
38             _counter.dec();
39         }
40     }
41 }
42 public class Race {
43     public static void main(String[] args) {
44         for(int i = 0; i < 100; i++) {
45             Counter cnt = new Counter(0);
46             IThread it = new IThread(cnt);
47             DThread dt = new DThread(cnt);
48             it.start();
49             dt.start();
50             try {
51                 it.join();
52                 dt.join();
53             } catch (InterruptedException e) {
54                 e.printStackTrace();
55             }
56             System.out.println(cnt.value());
57         }
58     }
59 }

```

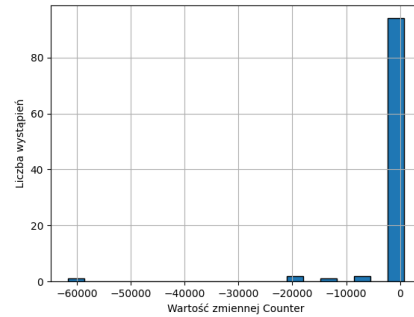
Kod Java 1: Wersja bez synchronizacji

4 Otrzymane wyniki dla wersji podstawowej

Znając operacje, którymi zajmuje się program wnioskujemy, że wartość zmiennej po operacjach dodawania i odejmowania powinna pozostać niezmienną, czyli równa 0. Jednak uwzględniając specyfikę wykonywania operacji na tym samym zasobie przez różne wątki nie mamy pewności czy zawsze otrzymamy poprawny wynik. Wykres (1) oraz (2) prezentują wartości zmiennej po obliczeniach. Doświadczenie zostało przeprowadzone 100 razy.



Rysunek 1: Wartość zmiennej dla kolejnych doświadczeń



Rysunek 2: Histogram dla 100 doświadczeń

Analizując oba wykresy zauważamy, że wartości zmiennej po obliczeniach jest przeważnie równa 0. Trzeba jednak zwrócić uwagę na odstępstwo od tej reguły przede wszystkim dla kilku początkowych cykli obliczeń. Może to być spowodowane rozgrzewaniem JVM oraz pamięcią Cache procesora, ponieważ miałem do czynienia z tym samym problemem za każdym razem uruchomienia programu.

5 Dodanie mechanizmu synchronizacji

W celu zapewnienia dobrego wyniku operacji zastosujemy algorytm Petersona [1]. Prosty algorytm zapewniający dostęp do sekcji krytycznej dla wielu wątków, w naszym przypadku tylko dwóch. W celu zrealizowania tego sposobu będziemy musieli zmodyfikować klasę Counter, dodać nową PetersonLock oraz wykonując funkcje `inc` oraz `dec` przesyłać informacje, który wątek ją wywołuje. Dodatkowo użyć zmiennych typu `volatile`, która sprawia że zmienna zawsze jest odczytywana z pamięci głównej a nie podręcznej wątków. Kod Java (2) zawiera zmiany dokonane na rzecz synchronizacji.

```

1 class Counter {
2     private int _val;
3     private PetersonLock lock = new PetersonLock();
4     public Counter(int n) {
5         _val = n;
6     }
7     public void inc(int threadId) {
8         lock.lock(threadId);
9         try {
10             _val++;
11         }finally {
12             lock.unlock(threadId);
13         }
14     }
15     public void dec(int threadId) {
16         lock.lock(threadId);
17         try {
18             _val--;
19         }finally {
20             lock.unlock(threadId);
21         }
22     }
23     public int value() {
24         return _val;
25     }
26 }
27 class PetersonLock {
28     private volatile boolean[] flag = new boolean[2];
29     private volatile int turn;
30     public void lock(int threadId) {
31         int otherId = 1 - threadId;

```

```

32     flag[threadId] = true;
33     turn = otherId;
34     while (flag[otherId] && turn == otherId) {
35         Thread.yield();
36     }
37 }
38 public void unlock(int threadId) {
39     flag[threadId] = false;
40 }
41 }
42 // Wątek, który inkrementuje licznik 100.000 razy
43 class IThread extends Thread {
44     private Counter _counter;
45     public IThread(Counter counter) {
46         _counter = counter;
47     }
48     @Override
49     public void run() {
50         for (int i = 0; i < 100_000; i++) {
51             _counter.inc(1);
52         }
53     }
54 }
55
56 // Wątek, który dekrementuje licznik 100.000 razy
57 class DThread extends Thread {
58     private Counter _counter;
59     public DThread(Counter counter) {
60         _counter = counter;
61     }
62     @Override
63     public void run() {
64         for (int i = 0; i < 100_000; i++) {
65             _counter.dec(0);
66         }
67     }
68 }

```

Kod Java 2: Uwzględnienie synchronizacji

Użycie algorytmu Petersona daje nam przewidywalność wyniku, który zgodnie z założeniami ma wynosić 0. Jednak powoduje on spowolnienie wykonania programu. Czas programu bez mechanizmów synchronizacji wynosił 35 ms natomiast dla programu implementującego algorytm Petersona 3147 ms. Jest to pogorszenie o blisko 90 razy.

6 Wnioski

Brak synchronizacji dostępu do zmiennej Counter prowadził do nieprzewidywalnego wyniku. Mielśmy do czynienia z typowym wyścigiem wątków. Dodanie mechanizmu synchronizacji w postaci algorytmu Petersona okazało się skutecznym sposobem wyeliminowania błędnych wartości końcowych zmiennej Counter. Nie możemy również pominąć aspektu wydajnościowego. Podstawowy program jest znacznie szybszy od rozwiązania implementującego mechanizm synchronizacji. Zastosowanie wbudowanych mechanizmów spowoduje zmniejszenie tej różnicy.

Literatura

- [1] Algorytm Petersona - https://en.wikipedia.org/wiki/Peterson%27s_algorithm
- [2] Synchronizacja wątków w Java - <https://www.artima.com/insidejvm/ed2/threadsynch.html>